



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems
Fall Semester 2026-2027

Department of Software and Systems Engineering
SWE3004 - Software Design and Development Project
Review -1

Date: 13.8.2026

RAG-Based College Helpdesk Chatbot Using Trusted Document Retrieval

22MIS0252 - Pradish G

Under the Guidance of

Dr. PRABHU J

Professor Grade 1

SCORE

Guide Signature with date

Dr. PHABHU J

Internal Examiner -1

Signature

Internal Examiner-2

Signature

ABSTRACT:

Students often need information from college documents like academic regulations, exam rules, scholarship details, and hostel policies. These documents are spread across multiple PDFs and web pages, making it hard to find specific information quickly. General-purpose chatbots like ChatGPT do not have access to college-specific documents and may give wrong answers.

This project builds an RAG-based chatbot for college helpdesk use. The system takes trusted college documents, splits them into smaller text chunks, converts them into vector embeddings, and stores them in a vector database. When a student asks a question, the system retrieves the most relevant chunks and uses an LLM to generate an answer based on that content. The source document is shown with each answer. If relevant information is not found, the system says so instead of guessing. As a future direction, the system may also use document metadata like version, authority, and effective date to handle conflicting or outdated information.

Keywords: Retrieval-Augmented Generation, RAG, college chatbot, document retrieval, vector database, LLM, question answering, information retrieval

1. INTRODUCTION:

Colleges maintain many official documents - academic regulations, exam rules, scholarship policies, hostel rules, certificate procedures, and circulars. Students need to search through these documents regularly, but finding the right information is slow and difficult. Websites may be outdated, PDFs are hard to search, and administrative offices have limited hours. General-purpose chatbots do not have access to a specific college's documents, so their answers to institution-specific questions may be wrong.

Retrieval-Augmented Generation (RAG) combines information retrieval with text generation. Instead of relying only on the LLM's training data, an RAG system retrieves relevant content from a set of documents and uses it to generate the answer. This project uses RAG to build a college helpdesk chatbot that answers questions based on official college documents, shows the source, and clearly indicates when the information is not available.

2. PROBLEM STATEMENT:

College students need accurate information from official documents (regulations, exam rules, circulars, etc.), but this information is scattered across multiple files and formats. Manually searching is time-consuming and students often rely on unofficial or outdated sources. General-purpose chatbots cannot answer college-specific questions reliably.

The proposed system solves this by building a RAG-based chatbot that retrieves relevant information from trusted college documents, generates grounded answers, shows the source reference, and avoids unsupported responses.

3. OBJECTIVES:

1. **Collect and organize** official college documents (regulations, exam rules, scholarships, circulars) into a structured knowledge base.
2. **Preprocess and chunk** documents by extracting text from PDF/DOCX, cleaning it, and splitting it into smaller segments.
3. **Generate vector embeddings** for each chunk using an embedding model and store them in ChromaDB.
4. **Build a retrieval pipeline** that converts student questions into vectors and retrieves the most relevant document chunks.
5. **Generate document-grounded answers** using an LLM with retrieved chunks as context, displaying the source reference.
6. **Handle unavailable information** by detecting insufficient evidence and informing the student instead of generating unsupported answers.

4. SCOPE OF THE PROJECT:

What the System Covers

- Academic queries (regulations, grading, CGPA, academic calendar)
- Examination queries (schedules, revaluation, supplementary exams)
- Certificate/procedure queries (Bonafide, TC, transcripts, no-dues)
- Scholarship queries (eligibility, application steps, deadlines)
- Hostel and general student information
- All answers are based only on uploaded college documents

What the System Does NOT Cover

- No transactions (fee payment, course registration, certificate generation)

- No real-time data from college ERP or student management systems
- No answers for topics not in the uploaded documents
- Does not replace administrative staff or official helpdesk

5. PROPOSED SYSTEM:

The system has two pipelines:

5.1 Document Ingestion Pipeline

1. **Document Collection** - Collect official college documents (PDF, DOCX)
2. **Text Extraction** - Extract text using PDF/DOCX parsers
3. **Text Preprocessing** - Clean text (remove repeated headers, formatting artifacts)
4. **Text Chunking** - Split into 500-1000 character chunks with overlap
5. **Embedding Generation** - Convert each chunk to a vector using an embedding model
6. **Vector Storage** - Store embeddings + text + metadata in ChromaDB

5.2 Query-Response Pipeline

1. **Student Question** - Student types a question in the React chat interface
2. **Query Embedding** - Convert the question into a vector using the same embedding model
3. **Retrieval Search** - ChromaDB for the top-k most similar document chunks
4. **LLM Generation** - Pass retrieved chunks + question to LLM to generate a grounded answer
5. **Answer + Source** - Display the answer with source document reference; if no relevant info found, indicate that clearly

6. LITERATURE SURVEY:

S. No	Title	Merits	Demerits
1	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)	Introduced RAG; combines retriever + generator; allows knowledge updates without retraining	Computationally expensive at inference; depends on quality of external knowledge source
2	Dense Passage Retrieval for Open-Domain QA	Dense retrieval outperforms BM25; captures semantic similarity beyond keywords	Needs large, annotated training data; may not generalize to specialized

	(Karpukhin et al., 2020)		domains
3	REALM: Retrieval-Augmented Language Model Pre-Training (Guu et al., 2020)	Joint pre-training of retriever + LM; unsupervised retriever learning; interpretable outputs	Complex training with asynchronous index updates; stale vector issues
4	Self-RAG: Learning to Retrieve, Generate, and Critique (Asai et al., 2023)	Dynamically decides when to retrieve; uses reflection tokens for self-evaluation	Adds token overhead; complex training procedure
5	Survey of Hallucination in NLG (Ji et al., 2023)	Comprehensive hallucination taxonomy; reviews detection and mitigation strategies	Survey only, no new solution; may become outdated quickly
6	Billion-Scale Similarity Search with GPUs (Johnson et al., 2019)	GPU-accelerated nearest neighbor search (FAISS); handles billion-scale vectors	Approximate methods trade off recall accuracy; needs GPU hardware
7	Sentence-BERT (Reimers & Gurevych, 2019)	Reduced semantic similarity computation from hours to seconds; widely used sentence-transformers library	Bi-encoder slightly less accurate than cross-encoder; varies across languages
8	RAG for Large Language Models: A Survey (Gao et al., 2023)	Categorized RAG into Naive/Advanced/Modular paradigms; structured overview	Limited coverage of domain-specific RAG applications
9	Fusion-in-Decoder for Open Domain QA (Izcard & Grave, 2021)	Performance improves with more retrieved passages; strong benchmark results	Gains plateau after certain number of passages; increases latency
10	BERT: Pre-training of Deep Bidirectional Transformers (Devlin et al., 2019)	Bidirectional pre-training; strong results on 11 NLP benchmarks; easy fine-tuning	Large model size; knowledge stored implicitly, hard to update

11	BM25 and Beyond (Robertson & Zaragoza, 2009)	Theoretical foundation for BM25; strong retrieval baseline with decades of validation	Cannot capture semantic similarity between different words with same meaning
12	Reading Wikipedia to Answer Open-Domain Questions (Chen et al., 2017)	DrQA combines TF-IDF retrieval + neural reader; demonstrated retrieve-then-read approach	TF-IDF retriever has limited semantic understanding; only factoid QA
13	Language Models as Knowledge Bases? (Petroni et al., 2019)	LAMA probe shows LMs store factual knowledge; works without supervision	Knowledge hard to update/verify; sensitive to query phrasing
14	Retrieval Augmentation Reduces Hallucination in Conversation (Shuster et al., 2021)	Empirically showed retrieval reduces hallucination in dialogue; confirmed by human evaluation	Tested on open-domain dialogue; not evaluated on domain-specific settings
15	RAGAS: Automated Evaluation of RAG (Es et al., 2024)	Automated RAG evaluation without ground-truth; metrics for retrieval + generation quality	Uses another LLM for evaluation (introduces bias); may miss domain-specific quality aspects
16	Query Rewriting in RAG LLMs (Ma et al., 2023)	Rewrite-Retrieve-Read framework; improves retrieval by rewriting queries	Adds latency; RL-based rewriter training adds complexity

6.1 FINDINGS IN LITERATURE SURVEY:

What existing research has done:

- RAG reduces hallucination and improves factual accuracy (Lewis et al., 2020; Shuster et al., 2021)
- Dense retrieval outperforms keyword-based retrieval for semantic matching (Karpukhin et al., 2020)
- RAGAS enables automated evaluation of RAG pipelines (Es et al., 2024)
- Query rewriting and self-reflection improve retrieval quality (Ma et al., 2023; Asai et al., 2023)

Gaps identified:

1. **Domain specificity** - Most RAG research uses Wikipedia. Limited work on institutional/college-specific document repositories.
2. **Document versioning** - Existing systems treat all documents as equally valid. No handling for outdated documents.
3. **Conflicting information** - No mechanism to detect conflicts when old and new documents say different things.
4. **Document authority** - No consideration of authority hierarchy (university regulation > departmental notice).
5. **Low-coverage hallucination** - LLMs may still hallucinate when retrieved content is only partially relevant.
6. **Institutional evaluation** - Existing evaluation frameworks don't consider source attribution correctness for institutional QA.

How our project addresses these: The project targets institutional document retrieval specifically. It uses only trusted college documents as the knowledge source and shows source references. As a future enhancement, the system plans to add document metadata (version, authority, effective date, status) for version-aware and conflict-aware retrieval. This is a planned direction, not yet implemented.

7. METHODOLOGY:

Current Implementation

1. **Requirement collection** - Identify common student queries and relevant documents
2. **Document collection** - Gather official college documents (PDF, DOCX)
3. **Text extraction** - Extract text using PyPDF2, python-docx
4. **Text cleaning** - Remove headers/footers, page numbers, formatting artifacts
5. **Text chunking** - Split into 500–1000 character overlapping chunks
6. **Embedding generation** - Generate embeddings using Sentence-Transformers
7. **Vector database creation** - Store embeddings + metadata in ChromaDB
8. **Query processing** - Accept question via React frontend, embed via same model
9. **Document retrieval** - Retrieve top-k similar chunks from ChromaDB

10. **Response generation** - Pass chunks + question to LLM; generate grounded answer

11. **Source display** - Show answer with source document reference

12. **Evaluation** - Assess quality using RAGAS or manual evaluation

Planned Advanced Features (Not Yet Implemented)

- Document metadata storage (version, effective date, authority, status)
- Version-aware retrieval (prefer latest active version)
- Authority-based ranking (university regulation > departmental notice)
- Conflict detection between documents
- Confidence-based response handling

8. SOFTWARE REQUIREMENTS: Functional and Non-Functional Requirements

Functional Requirement

1. Administrators can upload and manage college documents
2. System extracts text from PDF and DOCX files
3. System stores document chunks as vector embeddings
4. Students can ask questions through a chat interface
5. System retrieves relevant chunks and generates grounded answers
6. System shows source document reference with each answer
7. System indicates when information is not available

Non-Functional Requirements

1. Response time under 10 seconds
2. Simple, intuitive chat interface
3. Accessible via web browser
4. Handles concurrent users

Software and Tools

Component	Technology
Operating System	Windows / Linux
Programming Language	Python, JavaScript
Frontend	React
Backend	FastAPI (Python)
RAG Framework	LangChain
Embedding Model	Sentence-Transformers (e.g., all-MiniLM-L6-v2)
LLM	OpenAI GPT / Open-source LLM (to be finalized)
Vector Database	ChromaDB
Development Tools	VS Code, Postman
Version Control	Git / GitHub

9. SYSTEM ARCHITECTURE:

9.1 Document Ingestion Flow

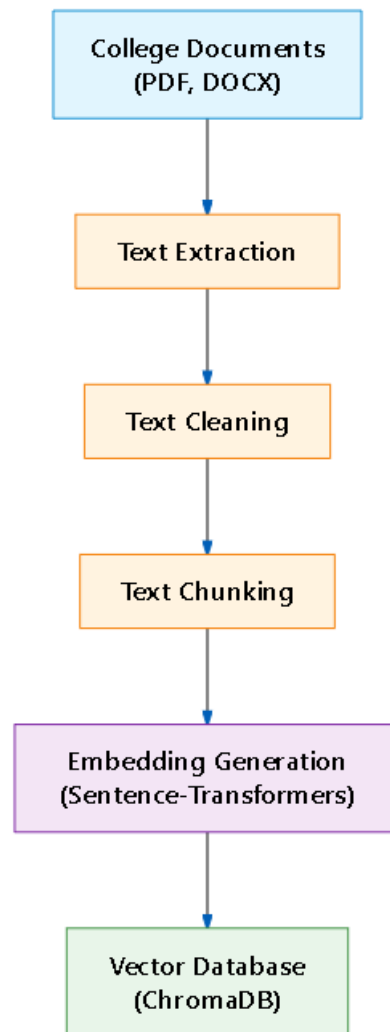


Fig. 9.1 Document Ingestion Pipeline

9.2 Query-Response Flow

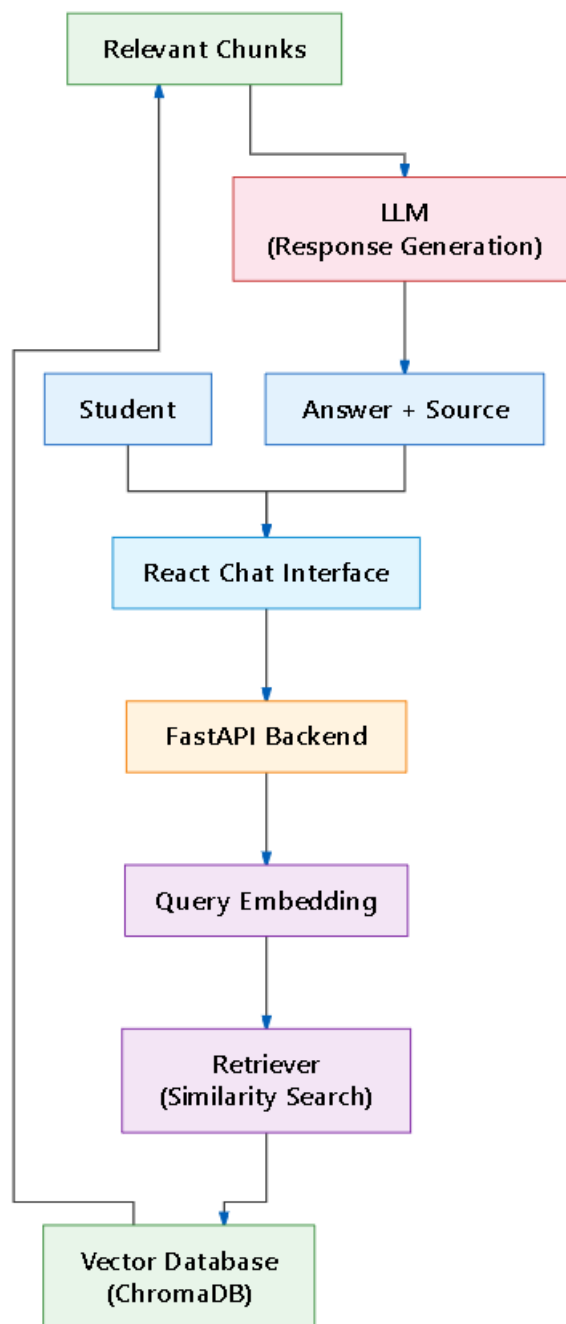


Fig. 9.2 Query-Response Pipeline

9.3 Overall System Architecture

Fig. 9.3 - System Architecture

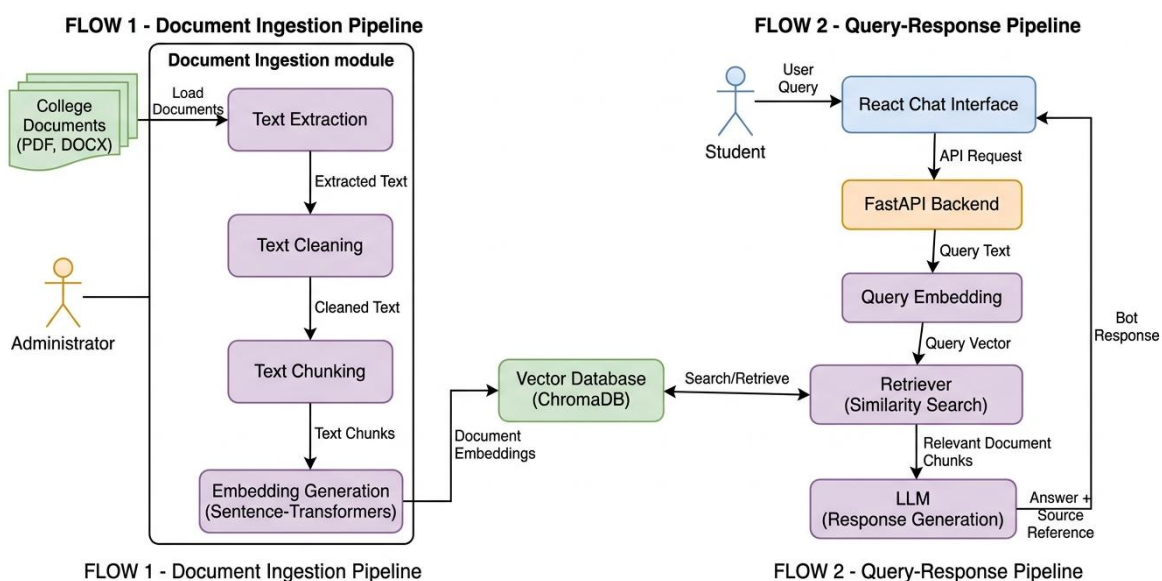


Fig. 9.3 Overall System Architecture

9.4 Document Metadata (Planned)

Field	Example
Document Name	Academic Regulations 2026
Authority	University / Department
Version	v2.1
Effective Date	01-07-2026
Status	Active / Superseded

This feature is planned and not yet implemented.

10. UML DIAGRAMS:

10.1 Use Case Diagram

Fig. 10.1 - Use Case Diagram

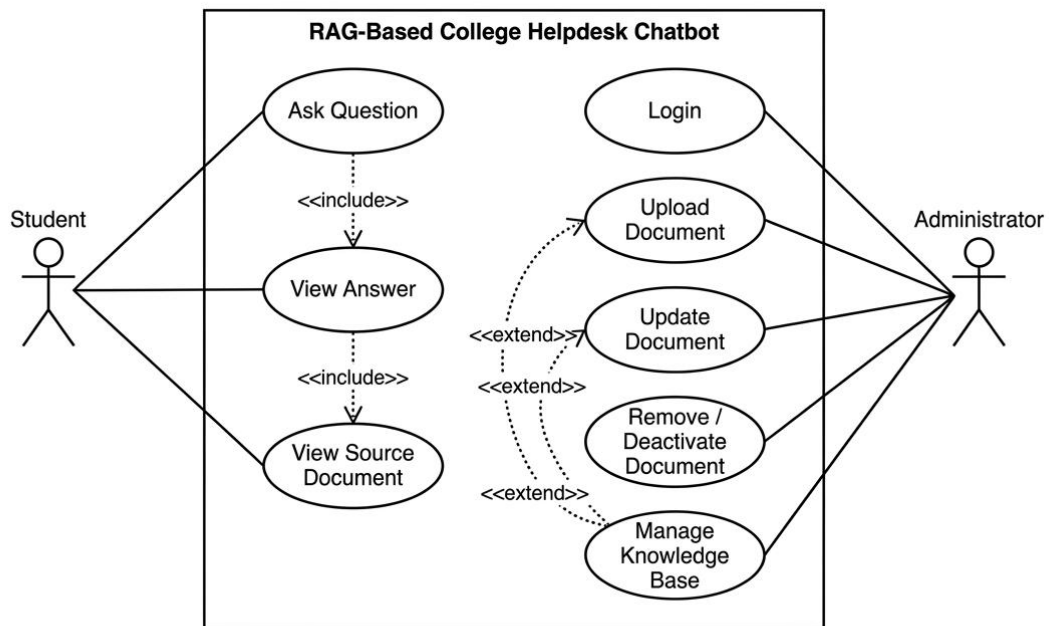


Fig. 10.1 Use Case Diagram

10.2 Class Diagram

Fig. 10.2 - Class Diagram

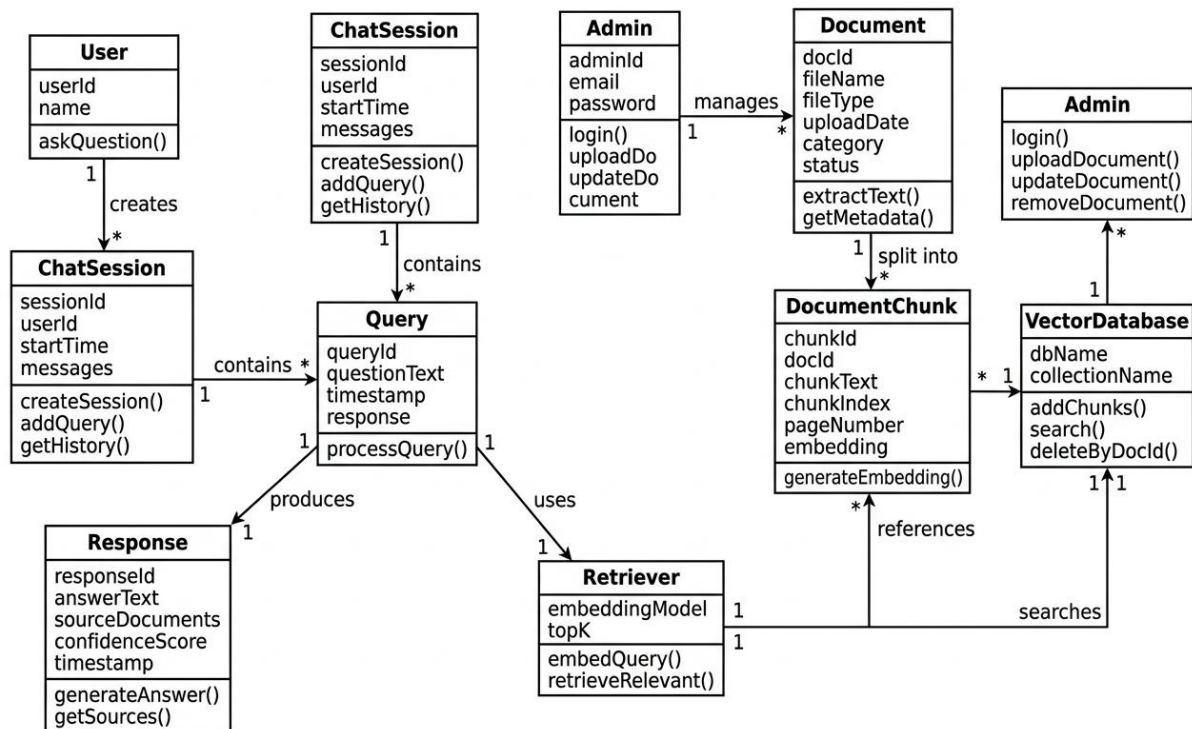


Fig. 10.2 Class Diagram

11. SUMMARY:

This project proposes a RAG-based college helpdesk chatbot that answers student questions using official college documents. The system extracts text from documents, creates vector embeddings, stores them in ChromaDB, and retrieves relevant chunks when a student asks a question. An LLM generates the answer based on retrieved content, and the source document is shown for verification.

A review of 16 research papers identified that while RAG effectively reduces hallucination, there is limited work on institutional document retrieval with version, authority, and conflict handling. As a future direction, the system plans to add document metadata for version-aware and conflict-aware retrieval.

REFERENCES:

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>
2. Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769–6781. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
3. Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. W. (2020). REALM: Retrieval-augmented language model pre-training. *Proceedings of the 37th International Conference on Machine Learning (ICML)*, PMLR 119, 3929–3938. <https://arxiv.org/abs/2002.08909>
4. Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *International Conference on Learning Representations (ICLR 2024)*. <https://arxiv.org/abs/2310.11511>
5. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1–38. <https://doi.org/10.1145/3571730>
6. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>
7. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>
8. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*. <https://arxiv.org/abs/2312.10997>
9. Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. *Proceedings of the 16th Conference of the*

- European Chapter of the Association for Computational Linguistics (EACL)*, 874–880. <https://doi.org/10.18653/v1/2021.eacl-main.74>
10. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
 11. Robertson, S. E., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389. <https://doi.org/10.1561/15000000019>
 12. Chen, D., Fisch, A., Weston, J., & Bordes, A. (2017). Reading Wikipedia to answer open-domain questions. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1870–1879. <https://doi.org/10.18653/v1/P17-1171>
 13. Petroni, F., Rocktäschel, T., Riedel, S., Lewis, P., Bakhtin, A., Wu, Y., & Miller, A. (2019). Language models as knowledge bases? *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2463–2473. <https://doi.org/10.18653/v1/D19-1250>
 14. Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). Retrieval augmentation reduces hallucination in conversation. *Findings of the Association for Computational Linguistics: EMNLP 2021*, 3784–3803. <https://doi.org/10.18653/v1/2021.findings-emnlp.320>
 15. Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2024): System Demonstrations*, 150–163. <https://arxiv.org/abs/2309.15217>
 16. Ma, X., Gong, Y., He, P., Zhao, H., & Duan, N. (2023). Query rewriting in retrieval-augmented large language models. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 5303–5315. <https://doi.org/10.18653/v1/2023.emnlp-main.322>